ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ & ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

Επεξεργασία Φωνής και Φυσικής Γλώσσας

2ο Εργαστήριο: Ανάλυση Συναισθήματος

Εμμανουήλ Χατζηγεωργίου 03122082
Αθανάσιος Φειδάκης 03122676

Αθήνα, 2026

Μέρος I
Προπαρασκευή

Ζητούμενο 1: Κωδικοποίηση Επισημειώσεων

Οι επισημειώσεις (labels) κωδικοποιούνται σε ακέραιους μέσω του `LabelEncoder` του scikit-learn, ο οποίος ταξινομεί αλφαβητικά τις μοναδικές ετικέτες και τους αποδίδει μηδενο-βασισμένους δείκτες. Η αντιστοίχιση εφαρμόζεται ταυτόχρονα στο training και test set, ώστε να είναι συνεπής.

```
DEBUG = True   # global flag -- set False to silence all debug prints

# EX1 -- label encoding
le = LabelEncoder()
le.fit(y_train)
y_train = le.transform(y_train)
y_test  = le.transform(y_test)

if DEBUG:
    print("Classes:", le.classes_)
    print("First␣10␣raw␣labels:", list(y_train_raw[:10]))
    print("First␣10␣encoded:␣␣␣", y_train[:10])
```

Παράδειγμα εξόδου για **MR**:

```
[MR] EX1 -- Label encoding
  Classes:              ['negative' 'positive']
  First 10 raw labels: ['positive', 'positive', 'positive', 'positive
     ',
                         'positive', 'positive', 'positive', 'positive
                            ',
                         'positive', 'positive']
  First 10 encoded:    [1 1 1 1 1 1 1 1 1 1]
```

Για το **Semeval2017A**: `negative` → 0, `neutral` → 1, `positive` → 2.

Ζητούμενο 2: Λεκτική Ανάλυση

Για τη λεκτική ανάλυση χρησιμοποιήθηκε ο `TweetTokenizer`, ο οποίος υπερτερεί του απλού διαχωρισμού βάσει κενού (`.split()`) καθώς αναγνωρίζει σύνθετες ενότητες όπως emoticons, hashtags και mentions ως ενιαία tokens. Η προσέγγιση αυτή είναι απαραίτητη για δεδομένα κοινωνικών δικτύων, όπου η στίξη και τα σύμβολα φέρουν έντονη συναισθηματική πληροφορία που θα χανόταν με έναν απλό διαχωρισμό. Στη συνέχεια, τα tokens μετατρέπονται σε αριθμητικούς δείκτες μέσω του λεξικού `word2idx`, με τις άγνωστες λέξεις να αντιστοιχίζονται στο token `<unk>`.

Τέλος, εφαρμόζεται padding και truncation με βάση την παράμετρο MAX_SENTENCE_LEN, ώστε να διασφαλιστεί η σταθερή διάσταση των διανυσμάτων εισόδου για την εκπαίδευση του νευρωνικού δικτύου.

```
# EX 2
tokenizer = TweetTokenizer()
self.data = [tokenizer.tokenize(sentence) for sentence in X]

self.embedded_data = [
    [word2idx.get(word, word2idx["<unk>"]) for word in sentence]
    for sentence in self.data
]

self.max_length = MAX_SENTENCE_LEN
self.embedded_data = [
    (s + [0] * self.max_length)[: self.max_length] for s in self.
       embedded_data
]
```

Τα δέκα πρώτα παραδείγματα μετά την επεξεργασία του TweetTokenizer εμφανίζονται παρακάτω:

```
=== EX2: Tokenization (first 10 sentences) ===
[0] ['05', 'Beat', 'it', '-', 'Michael', 'Jackson', '-', 'Thriller', '(', '25th', '
    Anniversary', 'Edition', ')', '[', 'HD', ']', 'http://t.co/A4K2B86PBv']
[1] ['Jay', 'Z', 'joins', 'Instagram', 'with', 'nostalgic', 'tribute', 'to', 'Michael', '
    Jackson', ':', 'Jay', 'Z', 'apparently', 'joined', 'Instagram', 'on', 'Saturday', 'and',
     '..', 'http://t.co/Qj9I4eCvXy']
[2] ['Michael', 'Jackson', ':', 'Bad', '25th', 'Anniversary', 'Edition', '(', 'Picture', '
    Vinyl', '):', 'This', 'unique', 'picture', 'disc', 'vinyl', 'includes', 'the', 'original
    ', '1', 'http://t.co/fKXhToAAuW']
[3] ['I', 'liked', 'a', '@YouTube', 'video', 'http://t.co/AaR3pjp2PI', 'One', 'Direction', '
    singing', '"', 'Man', 'in', 'the', 'Mirror', '"', 'by', 'Michael', 'Jackson', 'in', '
    Atlanta', ',', 'GA', '[', 'June', '26', ',']
[4] ['18th', 'anniv', 'of', 'Princess', "Diana's", 'death', '.', 'I', 'still', 'want', 'to',
     'believe', 'she', 'is', 'living', 'on', 'a', 'private', 'island', 'away', 'from', 'the
    ', 'public', '.', 'With', 'Michael', 'Jackson', '.']
[5] ['@oridaganjazz', 'The', '1st', 'time', 'I', 'heard', 'Michael', 'Jackson', 'sing', 'was
    ', 'in', 'Honolulu', ',', 'Hawaii', '@', 'a', 'restaurant', 'on', 'radio', '.', 'It', '
    was', 'A', '.', 'B', '.', 'C', '.', 'I', 'was', '13', '.', 'I', 'loved', 'it', '!']
[6] ["'", 'Michael', 'Jackson', "'", 'appeared', 'on', 'Saturday', '29', 'at', 'the', '9th',
     'place', 'in', 'the', 'Top', '20', 'of', "Miami's", 'Trends', ':', 'http://t.co/
    dXN2FWgUhb', '#trndnl']
[7] ['Are', 'you', 'old', 'enough', 'to', 'remember', 'Michael', 'Jackson', 'attending', '
    the', 'Grammys', 'with', 'Brooke', 'Shields', 'and', 'Webster', 'sat', 'on', 'his', 'lap
    ', 'during', 'the', 'show', '?']
[8] ['@etbowser', 'do', 'u', 'enjoy', 'his', '2nd', 'rate', 'Michael', 'Jackson', 'bit',
    '?', 'Honest', 'ques', '.', 'Like', 'the', "can't", 'feel', 'face', 'song', 'but', 'god
    ', "it's", 'so', 'obvious', 'they', 'want', 'MJ', '2.0']
[9] ['The', 'Weeknd', 'is', 'the', 'closest', 'thing', 'we', 'may', 'get', 'to', 'Michael',
    'Jackson', 'for', 'a', 'long', 'time', '...', 'especially', 'since', 'he', 'damn', 'near
    ', 'mimics', 'everything']
```

Παρατηρούμε ότι ο διαχωρισμός πραγματοποιείται με επιτυχία, καθώς ο tokenizer απομονώνει σωστά τα σημεία στίξης (π.χ. παρενθέσεις στο παράδειγμα [0]) και διατηρεί αυτούσιους τους συνδέσμους URL και τα hashtags (π.χ. '#trndnl' στο παράδειγμα [6]), στοιχεία που θα καταστρέφονταν με έναν απλό διαχωρισμό βάσει κενών.

Ζητούμενο 3: Κωδικοποίηση Παραδειγμάτων

Η μέθοδος **__getitem__** αποτελεί τη γέφυρα μεταξύ της αποθηκευμένης πληροφορίας και της εισόδου του μοντέλου. Η επιστροφή του πραγματικού μήκους (**length**) είναι κρίσιμη καθώς επιτρέπει στο δίκτυο να αγνοεί τα μηδενικά στοιχεία του padding κατά τον υπολογισμό των κρυφών καταστάσεων, αποφεύγοντας τη διάχυση θορύβου στη μνήμη του μοντέλου.

```
# EX3 -- Implementation of __getitem__
def __getitem__(self, index):
    example = torch.tensor(self.embedded_data[index], dtype=torch.
        long)
    label = self.labels[index]
    length = min(len(self.data[index]), self.max_length)
    return example, label, length
```

Παρακάτω παρουσιάζονται 5 παραδείγματα για επαλήθευση:

```
Example 0:
  Raw sentence: the rock is destined to be the 21st century's new " conan
     " and that he's going to make a splash even greater than arnold
     schwarzenegger , jean-claud van damme or steven segal .
  Dataset returns:
    example: [1, 1187, 16, 10506, 5, 31, 1, 4747, 1291147, 51, 9, 20525,
       9, 6, 13, 1291147, 227, 5, 166, 8, 17024, 157, 1441, 74, 5877,
       6836, 2, 1291147, 1425, 44003, 47, 4374, 26293, 3, 0, 0, 0, 0, 0,
       0]
    label:   1
    length:  34
------------------------------
Example 1:
  Raw sentence: the gorgeously elaborate continuation of " the lord of the
      rings " trilogy is so huge that a column of words cannot adequately
     describe co-writer/director peter jackson's expanded vision of j . r
     . r . tolkien's middle-earth .
  Dataset returns:
    example: [1, 72548, 5090, 10042, 4, 9, 1, 2631, 4, 1, 6932, 9, 12572,
       16, 99, 1361, 13, 8, 3216, 4, 1418, 1291147, 12288, 4611, 32164,
       205, 365, 1314, 1291147, 2770, 3187, 4, 5924, 3, 1866, 3, 1866, 3,
       1291147, 1291147]
    label:   1
    length:  40
------------------------------
Example 2:
  Raw sentence: effective but too-tepid biopic
  Dataset returns:
    example: [2000, 35, 1291147, 31493, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0]
    label:   1
    length:  4
------------------------------
Example 3:
  Raw sentence: if you sometimes like to go to the movies to have fun ,
```

```
     wasabi is a good place to start .
  Dataset returns:
    example: [84, 81, 1126, 118, 5, 247, 5, 1, 2596, 5, 33, 2888, 2,
       65642, 16, 8, 226, 234, 5, 473, 3, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0]
    label:   1
    length:  21
------------------------------
Example 4:
  Raw sentence: emerges as something rare , an issue movie that's so
     honest and keenly observed that it doesn't feel like one .
  Dataset returns:
    example: [13111, 20, 637, 2391, 2, 30, 505, 1055, 1291147, 99, 6000,
       6, 23325, 4251, 13, 21, 1291147, 983, 118, 49, 3, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
    label:   1
    length:  21
------------------------------
```

Θεωρητικός Σχολιασμός: Η μετατροπή σε `torch.long` είναι απαραίτητη καθώς οι δείκτες λέξεων χρησιμοποιούνται για το *embedding lookup*, μια πράξη που απαιτεί ακέραιες τιμές. Το *Zero-padding* διασφαλίζει ότι όλα τα δείγματα ενός batch έχουν τις ίδιες διαστάσεις, επιτρέποντας την παραλληλοποίηση των πράξεων.

Σχολιασμός επί των Αποτελεσμάτων:

- **Zero-Padding και Truncation:** Στο Παράδειγμα 2, η πρόταση έχει μόλις 4 tokens, οπότε συμπληρώνονται 36 μηδενικά στοιχεία για να φτάσει το `MAX_SENTENCE_LEN=40`. Αντίθετα, στο Παράδειγμα 1, η πρόταση φτάνει ακριβώς το όριο των 40 tokens, επιβεβαιώνοντας τη σταθερή διάσταση των εισόδων.

- **Διαχείριση Αγνώστων Λέξεων (OOV):** Παρατηρείται συχνά ο δείκτης `1291147`. Αυτός αντιστοιχεί στο token `<unk>`, υποδεικνύοντας λέξεις που δεν υπήρχαν στο αρχείο των προ-εκπαιδευμένων embeddings (π.χ. εξειδικευμένα ονόματα ή σύνθετοι όροι όπως *too-tepid*).

- **Διαχωρισμός Length και Padding:** Το `length` (π.χ. 34 στο Παράδειγμα 0) αναπαριστά το πληροφοριακό περιεχόμενο της πρότασης, το οποίο θα χρησιμοποιηθεί από το μοντέλο για να αγνοήσει το padding κατά την εκπαίδευση.

Ζητούμενο 4: Embedding Layer

Σε αυτό το στάδιο, υλοποιούμε το Embedding Layer του δικτύου `BaselineDNN`, το οποίο αναλαμβάνει τη μετατροπή των αριθμητικών δεικτών των λέξεων σε συνεχή διανύσματα σημασιολογικής αναπαράστασης.

```
# 1 - define the embedding layer
self.embedding = nn.Embedding(len(embeddings), len(embeddings[0]))
```

```
# 2 - initialize weights from pretrained word embeddings
self.embedding.weight = nn.Parameter(torch.Tensor(embeddings))

# 3 - freeze or finetune depending on trainable_emb
self.embedding.weight.requires_grad = trainable_emb
```

Αρχικοποιούμε το επίπεδο με προ-εκπαιδευμένα embeddings καθώς αυτά έχουν εκπαιδευτεί σε τεράστια σύνολα δεδομένων και κωδικοποιούν πλούσιες σημασιολογικές και συντακτικές σχέσεις μεταξύ των λέξεων. Η αρχικοποίηση με τα διανύσματα αυτά προσφέρει στο μοντέλο ένα σημείο εκκίνησης υψηλής ποιότητας ("ξέρει" ήδη ότι η λέξη "υπέροχο" είναι κοντά στη λέξη "καλό" πριν καν δει το δικό μας dataset), το οποίο θα ήταν αδύνατο να μάθει αυτόνομα χρησιμοποιώντας μόνο τα συγκριτικά μικρά σύνολα δεδομένων συναισθήματος που διαθέτουμε. Αυτό επιταχύνει την εκπαίδευση και βελτιώνει την ακρίβεια.

Κρατάμε παγωμένα τα βάρη (freeze=True) του επιπέδου κατά την εκπαίδευση κυρίως για να αποφύγουμε το overfitting. Αν το dataset μας είναι μικρό, η προσπάθεια να αλλάξουμε τα βάρη των embeddings μπορεί να καταστρέψει τις γενικές σημασιολογικές σχέσεις που ήδη υπάρχουν, προσαρμόζοντάς τες υπερβολικά στις ιδιαιτερότητες του συγκεκριμένου dataset. Επίσης, μειώνεται ο αριθμός των παραμέτρων που πρέπει να εκπαιδευτούν, κάνοντας τη διαδικασία πιο γρήγορη και λιγότερο απαιτητική σε πόρους.

Ζητούμενο 5: Output Layers

Σε αυτό το σημείο, ορίζουμε την αρχιτεκτονική της ταξινόμησης, χρησιμοποιώντας ένα ενδιάμεσο κρυφό επίπεδο με μη γραμμική ενεργοποίηση για την εξαγωγή πιο σύνθετων χαρακτηριστικών πριν την τελική προβολή στις κλάσεις εξόδου.

```
hidden_size = 50
self.fc1 = nn.Linear(emb_dim, hidden_size)
self.ac1 = nn.ReLU()
self.output_layer = nn.Linear(hidden_size, output_size)
```

Η σύνθεση γραμμικών μετασχηματισμών είναι και αυτή ένας γραμμικός μετασχηματισμός: $\mathbf{y} = A_2(A_1\mathbf{x}) = (A_2A_1)\mathbf{x}$. Το γινόμενο A_2A_1 αποτελεί έναν ενιαίο πίνακα, επομένως όσα γραμμικά επίπεδα και αν συνδέσουμε στη σειρά, το σύνολο των συναρτήσεων που μπορεί να αναπαραστήσει το μοντέλο παραμένει το ίδιο σύνολο γραμμικών συναρτήσεων - το ίδιο δηλαδή αν είχαμε ένα μόνο μοναδικό γραμμικό επίπεδο με λιγότερες παραμέτρους. Το βέλτιστο επιτεύξιμο σφάλμα παραμένει το ίδιο, καθώς προσθέτουμε πλεονάζουσες παραμέτρους χωρίς να επεκτείνουμε την εκφραστική ισχύ του μοντέλου.

Η εισαγωγή μιας μη γραμμικής ενεργοποίησης, όπως η ReLU, ανάμεσα στα επίπεδα καθιστά το δίκτυο ικανό πλέον να αναπαραστήσει μη γραμμικά όρια απόφασης, διευρύνοντας σημαντικά την εκφραστικότητά του και επιτρέποντας (πιθανώς) την επίτευξη χαμηλότερου σφάλματος κατά την εκπαίδευση. Χωρίς αυτήν, το δίκτυο θα μπορούσε να μοντελοποιήσει μόνο απλές, γραμμικές τάσεις, κάτι που δεν αρκεί για να κατανοήσει τη σημασιολογία της γλώσσας (π.χ. ειρωνεία ή σύνθετα νοήματα)

Ζητούμενο 6: Forward Pass

Σε αυτό το τελικό στάδιο του μοντέλου, ορίζουμε τη ροή των δεδομένων μέσα από το μοντέλο μας, υλοποιώντας τον υπολογισμό της διανυσματικής αναπαράστασης της πρότασης μέσω του μέσου όρου των embeddings, εξαιρώντας τα στοιχεία που προέκυψαν από padding.

```
# 1 - embed the words
embeddings = self.embedding(x)

# 2 - mean of word embeddings (excluding padding)
representations = torch.sum(embeddings, dim=1) \
                  / lengths.unsqueeze(1).float()

# 3 - non-linear transformation
representations = self.ac1(self.fc1(representations))

# 4 - project to classes
logits = self.output_layer(representations)
```

Το forward pass εκτελεί τα εξής βήματα:

$$E = \text{embed}(x) \qquad (B \times T \times d) \tag{1}$$

$$u = \frac{1}{N}\sum_{i=1}^{N} e_i \qquad \text{μέσος όρος embeddings} \tag{2}$$

$$r = \text{ReLU}(Wu + b) \qquad \text{μη-γραμμικός μετασχηματισμός} \tag{3}$$

$$\hat{y} = W_o r + b_o \qquad \text{προβολή στις κλάσεις} \tag{4}$$

Προσπαθώντας να δώσουμε μια διαισθητική ερμηνεία της αναπαράστασης αυτής, μπορούμε να πούμε ότι κάθε διάσταση του embedding space συλλαμβάνει μια αφηρημένη σημασιολογική ιδιότητα, επομένως ο μέσος όρος όλων των word embeddings σε μια πρόταση αντιπροσωπεύει το μέσο σημασιολογικό αναγνωριστικό/αποτύπωμα αυτής της πρότασης σε κάθε τέτοια διάσταση. Για παράδειγμα, έστω ότι μια διάσταση συσχετίζεται με την *ένταση του συναισθήματος*. Στην πρόταση *«ένα συναρπαστικό, εκπληκτικό αριστούργημα»*, οι λέξεις *συναρπαστικό*, *εκπληκτικό* και *αριστούργημα* προβάλλονται έντονα σε αυτόν τον άξονα, επομένως το κέντρο βάρους κληρονομεί μια υψηλή τιμή σε αυτή τη διάσταση - αποτυπώνοντας ότι η πρόταση ως σύνολο εκφράζει ισχυρό θετικό συναίσθημα.

Αδυναμίες της προσέγγισης του κέντρου βάρους.

- **Απώλεια της σειράς των λέξεων.** Η προσέγγιση αυτή αντιμετωπίζει το κείμενο ως Bag-of-Words. "Η ταινία δεν ήταν καλή, ήταν κακή" και "Η ταινία δεν ήταν κακή, ήταν καλή" θα έχουν σχεδόν τον ίδιο μέσο όρο, παρόλο που έχουν αντίθετο νόημα. Το κέντρο βάρους είναι δηλαδή αμετάβλητο σε μεταθέσεις των λέξεων εισόδου.

- **Αδυναμία χειρισμού άρνησης.** Κρίσιμο είναι ότι οι λέξεις άρνησης, όπως το "δεν" ή το "όχι", είναι συχνά σχεδόν ορθογώνιες προς την ιδιότητα της λέξης που αρνούνται. Εξετάζοντας την φράση "η ταινία ήταν ακριβή" έναντι του "η ταινία δεν ήταν ακριβή",

η λέξη δεν πιθανώς έχει σχεδόν μηδενική προβολή σε μια διάσταση κόστους, οπότε τα κέντρα βάρους των δύο προτάσεων καταλήγουν πολύ κοντά μεταξύ τους, παρά το γεγονός ότι φέρουν αντίθετα νοήματα.

- **Το context αγνοείται.** Κάθε λέξη λαμβάνει το ίδιο embedding ανεξάρτητα από το περιβάλλον της. Η λέξη "ακραία" παίρνει το ίδιο διάνυσμα είτε εμφανίζεται ως "ακραία καλό" είτε ως "ακραία κακό".

- **Ίση βαρύτητα σε όλες τις λέξεις.** Λέξεις χωρίς ιδιαίτερο νόημα (όπως "the", "is", "a") επηρεάζουν τον μέσο όρο εξίσου με λέξεις-κλειδιά (π.χ. "αριστούργημα", "αποτυχία"), θολώνοντας την τελική αναπαράσταση.

- **Αδυναμία διαχείρισης της Πολυσημίας.** Κάθε λέξη μπορεί να λαμβάνει διαφορετικά νοήματα ανάλογα με την πρόθεση του συντάκτη. Για παράδειγμα η λέξη "γλώσσα" μπορεί να αναφέρεται στο βιολογικό όργανο του ανθρώπου ή στο εργαλείο επικοινωνίας των ανρθώπων. Σε κάθε περίπτωση η αναπαράσταση θα είναι ίδια.

Ζητούμενο 7: Φόρτωση Παραδειγμάτων

Σε αυτό το στάδιο, οργανώνουμε τα δεδομένα σε mini-batches χρησιμοποιώντας την κλάση `DataLoader`, η οποία διαχειρίζεται την αποτελεσματική τροφοδοσία του μοντέλου και την ανακάτεμα των παραδειγμάτων.

```
train_loader = DataLoader(train_set, batch_size=BATCH_SIZE, shuffle=
   True)
test_loader  = DataLoader(test_set,  batch_size=BATCH_SIZE)
```

Μικρά vs Μεγάλα mini-batches

Το σφάλμα ανά batch $L_B = \frac{1}{|B|}\sum_{i\in B}\ell_i(\theta)$ είναι ένας δειγματικός μέσος όρος επί ανεξάρτητων και ισόνομων δειγμάτων, επομένως η διακύμανσή του είναι:

$$\mathrm{Var}(L_B) = \frac{\sigma_\ell^2}{|B|}$$

όπου σ_ℓ^2 είναι η διακύμανση του σφάλματος ανά δείγμα. Αυτή η διακύμανση αποτελεί ένδειξη της τραχύτητας της καμπύλης σφάλματος, με υψηλότερη διακύμανση να σημαίνει μεγαλύτερες διακυμάνσεις στο αναφερόμενο σφάλμα από batch σε batch. Καθώς αυτή μειώνεται με το μέγεθος $|B|$, εξηγείται γιατί:

- **Τα μικρά batches** έχουν υψηλό $\mathrm{Var}(L_B)$, επομένως η καμπύλη σφάλματος εμφανίζεται τραχιά. Ωστόσο, προσφέρουν περισσότερες ενημερώσεις βαρών ανά εποχή και ο θόρυβος των κλίσεων εμποδίζει το μοντέλο από το να παγιδευτεί σε απότομα τοπικά ελάχιστα, μειώνοντας την υπερπροσαρμογή.

- **Τα μεγάλα batches** έχουν χαμηλό $\mathrm{Var}(L_B)$, οδηγώντας σε μια ομαλή καμπύλη σφάλματος και πιο προβλέψιμη σύγκλιση. Το μειονέκτημα είναι οι λιγότερες ενημερώσεις ανά εποχή και η τάση σύγκλισης σε πιο απότομα ελάχιστα, τα οποία ενδέχεται να έχουν χειρότερη γενίκευση.

Shuffle mini-batches

Χωρίς shuffle, τα δεδομένα ενδεχομένως να είχαν μια συγκεκριμένη σειρά (π.χ. όλα τα θετικά σχόλια στην αρχή και όλα τα αρνητικά στο τέλος), επομένως το μοντέλο θα μάθαινε αυτή τη σειρά αντί για το πραγματικό νόημα των λέξεων. Αυτό εισάγει bias καθώς οι ενημερώσεις των βαρών στο τέλος κάθε εποχής θα καθοδηγούνταν συστηματικά από το ίδιο υποσύνολο παραδειγμάτων, στρέφοντας το μοντέλο προς τα μοτίβα αυτού του υποσυνόλου. Για παράδειγμα, αν τα τελευταία batches περιείχαν κυρίως αρνητικές κριτικές, οι τελικές ενημερώσεις κάθε εποχής θα ωθούσαν το μοντέλο προς την πρόβλεψη του αρνητικού. Το ανακάτεμα σπάει αυτή τη σταθερή σειρά, διασφαλίζει διαφορετικές συνθέσεις batches σε κάθε εποχή και καθιστά τις εκτιμήσεις των κλίσεων πιο αντιπροσωπευτικές του πλήρους συνόλου δεδομένων κατά τη διάρκεια της εκπαίδευσης. Το μοντέλο ωθείται να μάθει πιο robust χαρακτηριστικά και εμποδίζεται από την απομνημόνευση της σειράς των παραδειγμάτων, με αποτέλεσμα να γενικεύει αποτελεσματικά.

Ζητούμενο 8: Βελτιστοποίηση

Σε αυτό το στάδιο, ορίζουμε τη στρατηγική εκπαίδευσης επιλέγοντας την κατάλληλη συνάρτηση απώλειας βάσει του πλήθους των κλάσεων, απομονώνοντας τις εκπαιδεύσιμες παραμέτρους και επιλέγοντας έναν αλγόριθμο βελτιστοποίησης με προσαρμοστικό ρυθμό μάθησης.

```
if DATASET == "MR":
    criterion = nn.BCEWithLogitsLoss()
    output_dim = 1  # single logit for binary classification
elif DATASET == "Semeval2017A":
    criterion = nn.CrossEntropyLoss()
    output_dim = 3  # 3 classes: positive, negative, neutral

model = BaselineDNN(
    output_size=output_dim,
    embeddings=embeddings,
    trainable_emb=EMB_TRAINABLE,
)

parameters = [p for p in model.parameters() if p.requires_grad]
optimizer = torch.optim.Adam(parameters, lr=0.001)
```

Loss function. Για το σύνολο δεδομένων MR (2 κλάσεις) χρησιμοποιούμε τη `BCEWithLogitsLoss`, η οποία χρησιμοποιεί την συνάρτηση sigmoid. Το επίπεδο εξόδου έχει έναν μόνο νευρώνα, παράγοντας ένα logit. Η προβλεπόμενη κλάση είναι θετική εάν το logit υπερβαίνει το μηδέν. Για την εργασία Semeval2017A (3 κλάσεις) χρησιμοποιούμε τη `CrossEntropyLoss`, η οποία εφαρμόζει

εσωτερικά τη συνάρτηση softmax και υπολογίζει την αρνητική λογαριθμική πιθανοφάνεια της σωστής κλάσης.

Εκπαιδεύσιμες Παράμετροι Συλλέγουμε μόνο τις παραμέτρους με `requires_grad == True`. Όταν το embedding layer είναι παγωμένο (`trainable_emb=False`), τα βάρη του εξαιρούνται από τον βελτιστοποιητή, αφήνοντας μόνο τα βάρη του κρυφού επιπέδου και του επιπέδου εξόδου προς ενημέρωση. Αυτό αποφεύγει τη σπατάλη υπολογιστικών πόρων σε παγωμένες παραμέτρους και, το σημαντικότερο, διασφαλίζει ότι οι προ-εκπαιδευμένες παράμετροι του embedding layer δεν θα αλλοιωθούν από το μικρό μας σύνολο δεδομένων.

Optimizer. Χρησιμοποιούμε τον Adam, ο οποίος διατηρεί προσαρμοστικούς ρυθμούς μάθησης ανά παράμετρο μέσω τρεχουσών εκτιμήσεων της πρώτης και της δεύτερης ροπής της κλίσης. Αυτό τον καθιστά κατάλληλο για το πλαίσιό μας, όπου διαφορετικές ομάδες παραμέτρων (κρυφό επίπεδο έναντι επιπέδου εξόδου) μπορεί να έχουν πολύ διαφορετικά μεγέθη κλίσεων, εξαλείφοντας την ανάγκη για χειροκίνητο συντονισμό του του learning rate.

Ζητούμενο 9: Εκπαίδευση

Σε αυτό το τελικό στάδιο, υλοποιούμε τις συναρτήσεις `train_dataset` και `eval_dataset`, οι οποίες διαχειρίζονται τη ροή των δεδομένων μέσω του μοντέλου, τον υπολογισμό του σφάλματος και την ενημέρωση των βαρών μέσω του αλγορίθμου backpropagation.

```
# training/py -- train_dataset (core loop)
def train_dataset(_epoch, dataloader, model, loss_function, optimizer
    ):
    model.train()
    for inputs, labels, lengths in dataloader:
        inputs, labels, lengths = inputs.to(device), labels.to(device
            ), lengths.to(device)
        optimizer.zero_grad()
        outputs = model(inputs, lengths)
        if isinstance(loss_function, nn.BCEWithLogitsLoss):
            outputs, labels = outputs.squeeze(1), labels.float()
        loss_function(outputs, labels).backward()
        optimizer.step()

# training.py -- eval_dataset (core loop)
def eval_dataset(dataloader, model, loss_function):
    model.eval()
    y_pred, y_true = [], []
    with torch.no_grad():
        for inputs, labels, lengths in dataloader:
            inputs, labels, lengths = inputs.to(device), labels.to(
                device), lengths.to(device)
            outputs = model(inputs, lengths)
            if isinstance(loss_function, nn.BCEWithLogitsLoss):
                outputs, labels = outputs.squeeze(1), labels.float()
                predicted = (outputs > 0).long()
```

```
            else:
                predicted = torch.argmax(outputs, dim=1)
            y_pred.append(predicted.cpu().numpy())
            y_true.append(labels.cpu().numpy())
```

Σχολιασμός Υλοποίησης: Και οι δύο συναρτήσεις μεταφέρουν τους tensors στη συσκευή του μοντέλου (CPU ή GPU) πριν από τον υπολογισμό. Στην περίπτωση της `BCEWithLogitsLoss`, η έξοδος συμπιέζεται από τη μορφή $(B, 1)$ σε $(B,)$ και οι ετικέτες μετατρέπονται σε `float`, καθώς η συγκεκριμένη συνάρτηση απώλειας απαιτεί float στόχους. Οι προβλέψεις βασίζονται στο πρόσημο του logit (κατώφλι στο 0) για τη δυαδική ταξινόμηση και στη συνάρτηση `argmax` για την ταξινόμηση πολλαπλών κλάσεων. Στη φάση της αξιολόγησης, η χρήση του `torch.no_grad()` είναι απαραίτητη για την εξοικονόμηση μνήμης, καθώς δεν απαιτείται η διατήρηση των κλίσεων.

Ζητούμενο 10: Αξιολόγηση

Σε αυτή την ενότητα παρουσιάζουμε τις επιδόσεις του βασικού μοντέλου `BaselineDNN` στα δύο σύνολα δεδομένων, καθώς και τις αντίστοιχες καμπύλες εκπαίδευσης που απεικονίζουν τη μεταβολή του σφάλματος ανά εποχή.

Σύνολο Δεδομένων MR (δυαδική ταξινόμηση: θετικό / αρνητικό):

Μετρική	Σκορ
Accuracy	0.701 ± 0.006
Recall (macro)	0.702 ± 0.006
F1 Score (macro)	0.701 ± 0.006

Σύνολο Δεδομένων Semeval2017A (3 κλάσεις: θετικό / αρνητικό / ουδέτερο):

Μετρική	Σκορ
Accuracy	0.564 ± 0.003
Recall (macro)	0.557 ± 0.004
F1 Score (macro)	0.535 ± 0.005

Οι καμπύλες εκπαίδευσης (σφάλμα εκπαίδευσης και δοκιμής ανά εποχή) για κάθε σύνολο δεδομένων παρουσιάζονται παρακάτω.

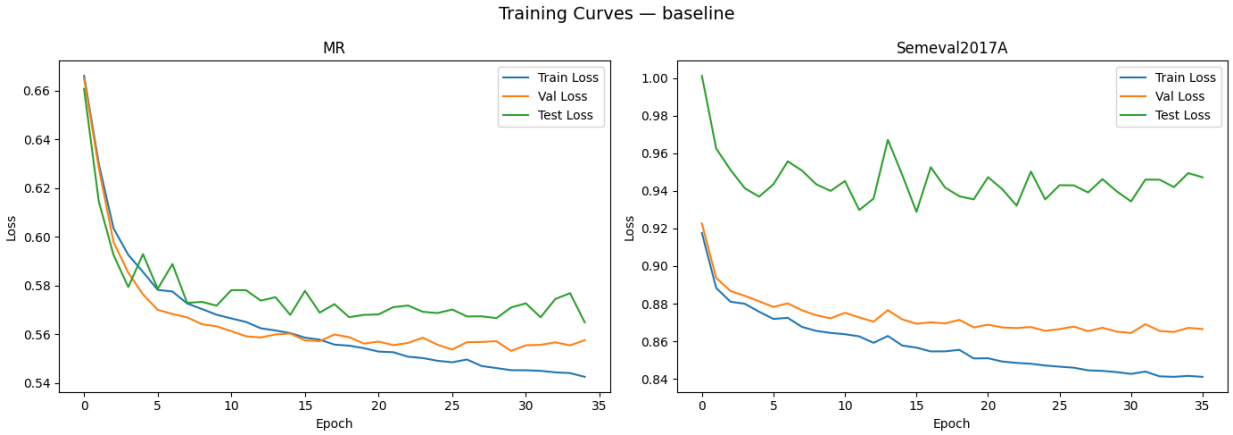


Figure 1: Καμπύλες εκπαίδευσης του Baseline DNN (mean pooling) στο MR (αριστερά) και Semeval2017A (δεξιά).

Σχολιασμός Διαφοράς Επιδόσεων

Η αξιοσημείωτη διαφορά στις επιδόσεις μεταξύ των δύο συνόλων δεδομένων (MR: ∼70% έναντι Semeval: ∼56%) ερμηνεύεται από τους εξής παράγοντες:

- **Πολυπλοκότητα Ταξινόμησης:** Το MR αποτελεί πρόβλημα δυαδικής ταξινόμησης (2 κλάσεις), όπου η τυχαία πρόβλεψη έχει πιθανότητα 50%. Αντίθετα, το Semeval περιλαμβάνει 3 κλάσεις (θετικό, αρνητικό, ουδέτερο), ανεβάζοντας τον βαθμό δυσκολίας καθώς η τυχαία πρόβλεψη πέφτει στο 33.3%.
- **Η "Θορυβώδης" Ουδέτερη Κλάση:** Η προσθήκη της ουδέτερης κλάσης στο Semeval εισάγει μεγάλη σημασιολογική ασάφεια. Τα ουδέτερα κείμενα συχνά περιέχουν λέξεις που υπάρχουν και στις πολωμένες κλάσεις, καθιστώντας τον διαχωρισμό τους μέσω ενός απλού μέσου όρου εξαιρετικά δύσκολο.
- **Φύση των Δεδομένων:** Το MR βασίζεται σε δομημένες κριτικές ταινιών με σχετικά καθαρό λεξιλόγιο. Το Semeval χρησιμοποιεί δεδομένα από το Twitter, τα οποία χαρακτηρίζονται από έντονο θόρυβο (ανορθογραφίες, αργκό, συντομογραφίες και σαρκασμό), στοιχεία που δυσχεραίνουν την αποτελεσματικότητα των στατικών προ-εκπαιδευμένων embeddings.

Σημείωση για τη Διαδικασία Εκπαίδευσης

Για τη βελτιστοποίηση της εκπαίδευσης χρησιμοποιήθηκε ένα ξεχωριστό σύνολο επικύρωσης (**validation set**), το οποίο αποτελεί το 20% των δεδομένων εκπαίδευσης. Η εκπαίδευση δεν εκτελέστηκε απαραίτητα για όλες τις 50 εποχές, αλλά χρησιμοποιήθηκε η τεχνική **Early Stopping**. Σύμφωνα με αυτή, η διαδικασία διακόπτεται αυτόματα εάν το σφάλμα στο σύνολο επικύρωσης δεν παρουσιάσει βελτίωση για 5 συνεχόμενες εποχές (**patience=5**). Όπως παρατηρούμε, το μοντέλο στο MR τρέχει κατά μέσο όρο 27 εποχές και στο Semeval 36, αποτρέποντας έτσι την υπερπροσαρμογή (*overfitting*) και διασφαλίζοντας ότι διατηρούνται τα βάρη που πέτυχαν τη γενικευμένη βέλτιστη απόδοση.

Ζητούμενο 11: Κατηγοριοποίηση με χρήση LLM

Σε αυτό το στάδιο, αξιολογήσαμε την ικανότητα του ChatGPT στην ταξινόμηση συναισθήματος μέσω zero-shot prompting, συγκρίνοντας την απόδοσή του με τα προηγούμενα νευρωνικά μοντέλα.

MR Dataset (20 Positive / 20 Negative)

Για το σύνολο δεδομένων MR, χρησιμοποιήθηκε η στρατηγική ανάθεσης στο μοντέλο του ρόλου του "expert movie critic and sentiment analyzer". Η προσέγγιση αυτή επέτρεψε στο μοντέλο να εστιάσει στο εξειδικευμένο λεξιλόγιο των κριτικών κινηματογράφου.

Αποτέλεσμα: Accuracy $= 1.0$ **(40/40 δείγματα)**

Η ανάλυση των αποφάσεων του μοντέλου αποκάλυψε τα εξής:

- **Εύκολες Περιπτώσεις:** Το μοντέλο εντόπισε κυρίαρχα tokens όπως το "masterpiece" ή την έμφαση μέσω επανάληψης ("bad, bad, bad") και προσδιοριστών έντασης ("just plain bad").

- **Δύσκολες Περιπτώσεις:** Σε αντίθεση με απλούς ταξινομητές, το LLM αναγνώρισε ότι αντιθετικοί σύνδεσμοι όπως το "but" ή το "even though" μετατοπίζουν το σημασιολογικό βάρος στο δεύτερο σκέλος της πρότασης. Χαρακτηριστικό παράδειγμα η φράση *"interesting, but not compelling"*, όπου το μοντέλο αντιλήφθηκε ότι ο περιορισμός του but αναιρεί τη θετική χροιά της πρώτης λέξης interesting.

Semeval2017A Dataset (20 Positive / 20 Negative / 20 Neutral)

Στα δεδομένα των κοινωνικών δικτύων, το μοντέλο έλαβε τον ρόλο του "expert social media sentiment analyzer", καθώς η γλώσσα του Twitter περιλαμβάνει αργκό, emojis και συντομογραφίες.

Αποτέλεσμα: Accuracy $= 1.0$ **(60/60 δείγματα).**

Τα κύρια συμπεράσματα από αυτή τη δοκιμή ήταν:

- **Κατανόηση Σαρκασμού:** Το μοντέλο εντόπισε με ακρίβεια τον σαρκασμό σε φράσεις όπως *"Thanks for nothing, Delta"*, ερμηνεύοντας σωστά το ειρωνικό πλαίσιο παρά την παρουσία της θετικής λέξης "Thanks".

- **Η Δυσκολία του Neutral:** Η κατηγορία αυτή αναδείχθηκε ως η πλέον απαιτητική. Το μοντέλο βασίστηκε στην απουσία συναισθηματικών επιθέτων και στη χρήση πραγματολογικών ρημάτων (π.χ. "announced", "rescheduled") που υποδηλώνουν μεταφορά πληροφορίας.

- **Εξωλεκτικά Στοιχεία:** Η χρήση emojis και στίξης λειτούργησε ως ενισχυτής της βεβαιότητας του μοντέλου για την αναγνώριση του συναισθήματος.

Συμπέρασμα και Σύγκριση

Η ανωτερότητα του LLM (100% accuracy έναντι ∼70% του Baseline στο MR και ∼53% F1 στο Semeval) οφείλεται στο ότι δεν λειτουργεί ως "bag-of-words" ταξινομητής. Ενώ το βασικό μας μοντέλο (BaselineDNN) επηρεάζεται από την παρουσία μεμονωμένων θετικών/αρνητικών λέξεων, το LLM αξιοποιεί **contextualized embeddings**. Αυτό του επιτρέπει να λειτουργεί ως αναλυτής λόγου, αντιλαμβανόμενο πώς η σύνταξη και το πλαίσιο (context) αλλοιώνουν τη σημασία των λέξεων.

Σημείωση: Το πλήρες ιστορικό των συνομιλιών με το μοντέλο και οι αναλυτικές αποκρίσεις του παρατίθενται στα επισυναπτόμενα αρχεία `chat_MR.txt` *και* `chat_SV.txt`*. Τα αρχεία αυτά βρίσκονται στο φάκελο prep.*

Μέρος II
Κυρίως Εργαστήριο

Ερώτημα 1: Mean + Max Pooling

1.1 Υλοποίηση

Η αναπαράσταση κάθε πρότασης υπολογίζεται πλέον ως η συνένωση του mean pooling και του max pooling των word embeddings:

$$\mathbf{u} = \bar{e} \,\|\, e_{\max}, \quad \bar{e} = \text{mean}(E), \quad e_{\max} = \max(E)$$

Ο constructor του `BaselineDNN` δέχεται ένα επιπλέον boolean όρισμα `max_concat`. Όταν είναι ενεργό, η πρώτη γραμμική στρώση δέχεται $2 \times d$ διαστάσεις αντί d:

```
# models.py -- BaselineDNN.__init__
self.fc1 = nn.Linear((2 if max_concat else 1) * emb_dim, hidden_size)
self.max_concat = max_concat
```

Στο `forward()`, το max pooling υπολογίζεται και συνενώνεται με το mean pooling:

```
# models.py -- BaselineDNN.forward
representations = torch.sum(embeddings, dim=1) / lengths.unsqueeze(1)
   .float()
if self.max_concat:
    max_emb = torch.max(embeddings, dim=1).values
    representations = torch.cat((representations, max_emb), dim=1)
```

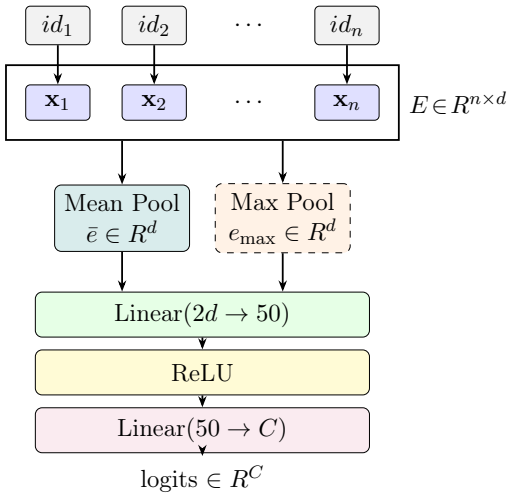


Figure 2: Αρχιτεκτονική `BaselineDNN` με mean+max pooling (d: διάσταση embedding, C: αριθμός κλάσεων). Το πλαίσιο E αγκαλιάζει ολόκληρη την ακολουθία. Ο κόμβος με διακεκομμένο πλαίσιο είναι η προσθήκη του Ερωτήματος 1 — στο baseline χρησιμοποιείται μόνο το mean pooling.

1.2 Σύγκριση και Αποτελέσματα

Το mean pooling αποτυπώνει τον "γενικό τόνο" της πρότασης, αλλά αδυνατεί να διατηρήσει ισχυρά τοπικά σήματα: ένα token με έντονη ενεργοποίηση σε κάποια διάσταση αραιώνεται από τον μέσο όρο. Το max pooling εξάγει, ανά διάσταση, τη μέγιστη τιμή μεταξύ όλων των tokens, διατηρώντας τα "ακραία" αυτά σήματα. Έτσι, η συνένωση των δύο δίνει στον γραμμικό ταξινομητή τόσο τη γενική εικόνα όσο τα κυρίαρχα χαρακτηριστικά.

Τα αριθμητικά αποτελέσματα που παρουσιάζονται σε όλα τα ερωτήματα προκύπτουν από τον μέσο όρο **15** ανεξάρτητων εκτελέσεων. Οι καμπύλες εκπαίδευσης απεικονίζουν ενδεικτικά την τελευταία από αυτές τις εκτελέσεις.

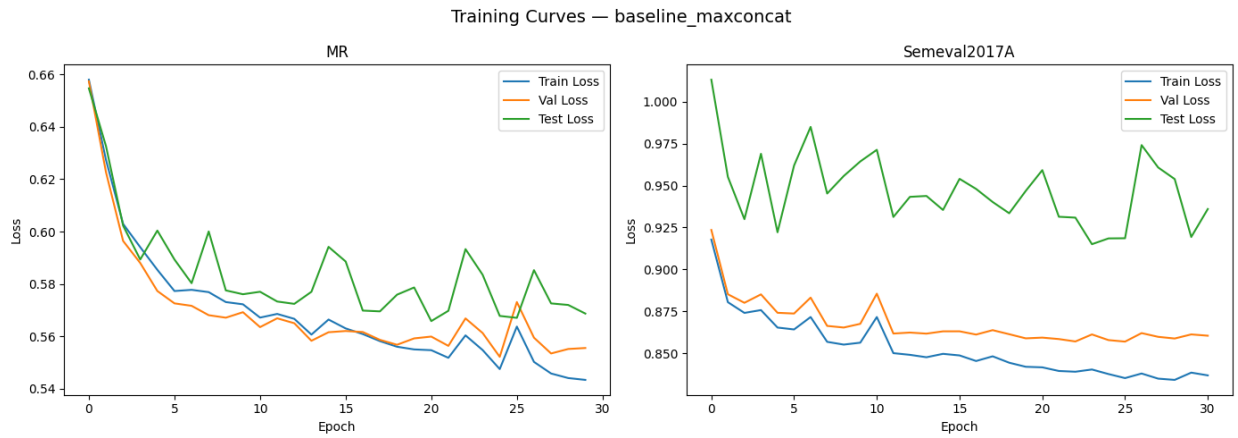


Figure 3: Καμπύλες εκπαίδευσης DNN με mean+max pooling στο MR (αριστερά) και Semeval2017A (δεξιά).

```
Dataset: MR
  reps: 15
  avg_epochs_run: 25.7/50
  accuracy: 0.6925478348 +/- 0.0064765096
  recall: 0.6928933506 +/- 0.0065627609
  f1-score: 0.6924116705 +/- 0.0064576630

Dataset: Semeval2017A
  reps: 15
  avg_epochs_run: 27.1/50
  accuracy: 0.5695376099 +/- 0.0042700593
  recall: 0.5637081899 +/- 0.0033647669
  f1-score: 0.5437162405 +/- 0.0068320880
```

Ερώτημα 2: LSTM & BiLSTM

2.1 Διαχωρισμός Εκπαίδευσης/Επαλήθευσης και Early Stopping

Σε όλα τα πειράματα (από το baseline μοντέλο και εφεξής) χρησιμοποιούμε *early stopping* για να αποφύγουμε το overfitting. Η τεχνική απαιτεί ένα ξεχωριστό *validation set* που κρατάμε εκτός εκπαίδευσης και παρακολουθούμε την απώλειά του σε κάθε εποχή. Όταν δεν παρατηρείται βελτίωση για `PATIENCE` συνεχόμενες εποχές ως προς το καλύτερο αποτέλεσμα ώς τη στιγμή εκείνη, η εκπαίδευση διακόπτεται και φορτώνεται αυτό το βέλτιστο checkpoint.

Αξίζει να σημειωθεί ότι δεν μπορούμε να χρησιμοποιήσουμε το *test set* για τις αποφάσεις early stopping. Αν το κάναμε, θα εκθέταμε έμμεσα πληροφορία του test set στη διαδικασία επιλογής μοντέλου, οδηγώντας σε τεχνητά βελτιωμένα αποτελέσματα. Αυτός είναι ο λόγος που στις καμπύλες εκπαίδευσης παρατηρούμε συνήθως:

$$\mathcal{L}_{\text{train}} < \mathcal{L}_{\text{val}} < \mathcal{L}_{\text{test}}$$

Το μοντέλο ελαχιστοποιεί ενεργά την απώλεια στα training data σε κάθε βήμα, οπότε το train loss τείνει να είναι χαμηλό. Το validation set επηρεάζει έμμεσα την εκπαίδευση μέσω των αποφάσεων early stopping, με αποτέλεσμα το validation loss να είναι υψηλότερο. Το test set δεν επηρεάζει καθόλου τη διαδικασία, οπότε αντικατοπτρίζει την πραγματική γενίκευση και αναμένεται να είναι ελαφρώς υψηλότερο ακόμα και από το validation loss. Καθώς ο ρόλος των training data στη μάθηση του μοντέλου είναι πολύ πιο θεμελιώδης από την ακριβή εύρεση του βέλτιστου σημείου early stopping, δεσμεύουμε μόνο το 20% του training set για validation, ώστε το μεγαλύτερο μέρος να παραμένει διαθέσιμο για εκπαίδευση.

```
# training.py -- torch_train_val_split
train_loader, val_loader = torch_train_val_split(
    train_set, BATCH_SIZE, BATCH_SIZE, val_size=VAL_SIZE
)
stopper = EarlyStopper(model, save_path, patience=PATIENCE)

for epoch in range(1, epochs + 1):
    train_dataset(epoch, train_loader, model, criterion, optimizer)
    val_loss, _ = eval_dataset(val_loader, model, criterion)
    if stopper.early_stop(val_loss):
        break

model.load_state_dict(torch.load(save_path))  # restore best
   checkpoint
```

Η `EarlyStopper` αποθηκεύει αυτόματα το checkpoint κάθε φορά που το validation loss βελτιώνεται, ώστε μετά τη διακοπή να φορτωθεί η βέλτιστη έκδοση και όχι η τελευταία.

2.2 LSTM

Το LSTM, σε αντίθεση με τον μέσο όρο embeddings, επεξεργάζεται τις λέξεις x_i *σειριακά* διατηρώντας δύο καταστάσεις: το *hidden state* h_i, που αποτελεί επίσης την έξοδο κάθε βήματος και λειτουργεί ως βραχυπρόθεσμη μνήμη, και το *cell state* c_i, που διαδίδεται από unit σε unit

μέσα στην αλυσίδα του LSTM ως μακροπρόθεσμη μνήμη, χωρίς να εκτίθεται ως έξοδος στα layers πάνω από αυτό.

Ο μηχανισμός θυρών (forget, input, output gates) ρυθμίζει ποια πληροφορία διατηρείται ή απορρίπτεται στις καταστάσεις h_i, c_i, επιτρέποντας στο δίκτυο να μαθαίνει μακρινές εξαρτήσεις χωρίς το vanishing gradient των απλών RNNs. Χρησιμοποιούμε ως αναπαράσταση κειμένου την τελευταία *πραγματική* έξοδο h_N, όπου N είναι το πραγματικό μήκος της πρότασης (όχι το padded μήκος). Τα πραγματικά μήκη υπολογίστηκαν στο Ζητούμενο 3 της προπαρασκευής και χρησιμοποιούνται εδώ για να επιλέξουμε το σωστό timestep:

```
# models.py -- LSTM.forward
X = torch.nn.utils.rnn.pack_padded_sequence(
    embeddings, lengths.cpu(), batch_first=True, enforce_sorted=False
)
ht, _ = self.lstm(X)
ht, _ = torch.nn.utils.rnn.pad_packed_sequence(ht, batch_first=True)

# pick the actual last token, not the padded end
representations = ht[torch.arange(batch_size), lengths - 1]
logits = self.linear(representations)
return logits
```

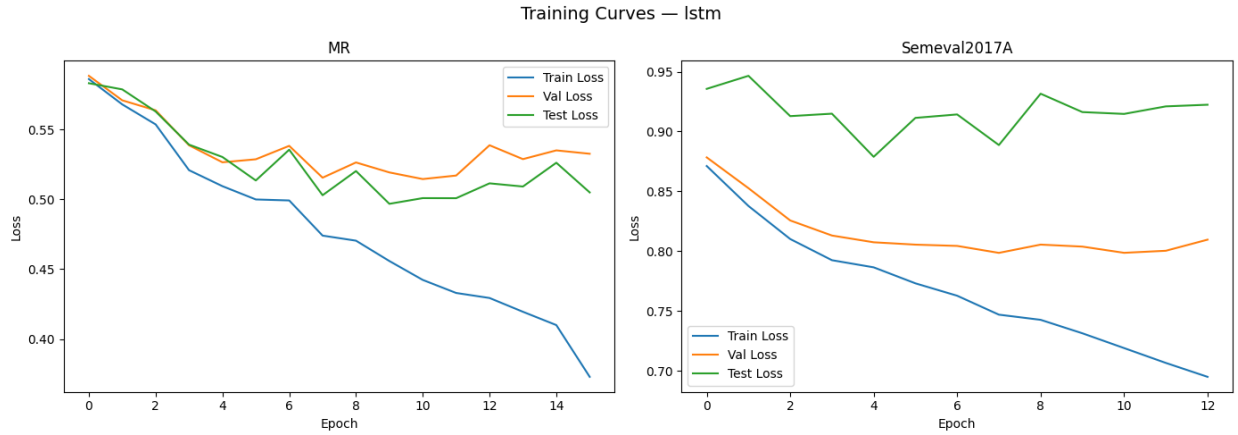


Figure 4: Καμπύλες εκπαίδευσης LSTM στο MR (αριστερά) και Semeval2017A (δεξιά).

```
Dataset: MR
  reps: 15
  avg_epochs_run: 16.7/50
  accuracy: 0.7524672709 +/- 0.0056484607
  recall: 0.7537708284 +/- 0.0060140794
  f1-score: 0.7521524026 +/- 0.0056182377

Dataset: Semeval2017A
  reps: 15
  avg_epochs_run: 14.4/50
```

```
  accuracy: 0.5921089764 +/- 0.0061172409
  recall: 0.5882900128 +/- 0.0068005036
  f1-score: 0.5711336058 +/- 0.0084290968
```

2.3 Αμφίδρομο LSTM (BiLSTM)

Το BiLSTM εκτελεί δύο LSTM παράλληλα: ένα από αριστερά προς τα δεξιά και ένα από δεξιά προς τα αριστερά. Η αναπαράσταση κάθε θέσης i είναι η συνένωση των δύο hidden states:

$$h_i = [\,\overrightarrow{h}_i \,\|\, \overleftarrow{h}_i\,], \quad h_i \in R^{2H}$$

Η λειτουργία αυτή υποστηρίζεται από την PyTorch θέτοντας την παράμετρο `bidirectional` κατά την κατασκευή του LSTM module.

Για να έχουμε αναπαράσταση με πλήρες πλαίσιο και από τις δύο κατευθύνσεις, επιλέγουμε συγκεκριμένα $\overrightarrow{h}_N$ (το μπροστινό LSTM στη θέση N, που έχει δει ολόκληρη την ακολουθία $1 \to N$) και $\overleftarrow{h}_1$ (το ανάστροφο LSTM στη θέση 1, που έχει δει $N \to 1$). Χρειάζεται λοιπόν να διπλασιάσουμε το μέγεθος της αναπαράστασης που περνάει από τον τελικό γραμμικό μετασχηματισμό.:

```
# models.py -- LSTM.__init__
self.representation_size = (
    2 * self.hidden_size if self.bidirectional else self.hidden_size
)

# models.py -- LSTM.forward
if self.bidirectional:
    forward_last  = ht[torch.arange(batch_size), lengths - 1, :self.
       hidden_size]
    backward_first = ht[:, 0, self.hidden_size:]
    representations = torch.cat([forward_last, backward_first], dim
       =1)
else:
    representations = ht[torch.arange(batch_size), lengths - 1]
```

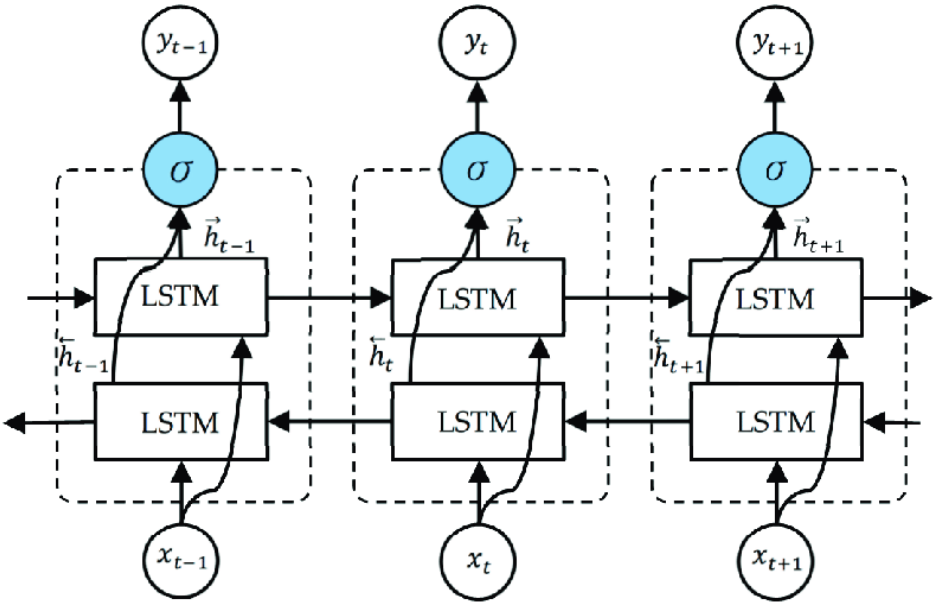


Figure 5: Αρχιτεκτονική BiLSTM. Η τελική αναπαράσταση είναι η συνένωση $[\overrightarrow{h}_N \parallel \overleftarrow{h}_1] \in R^{2H}$, που περνά από γραμμικό layer $R^{2H} \to R^C$.

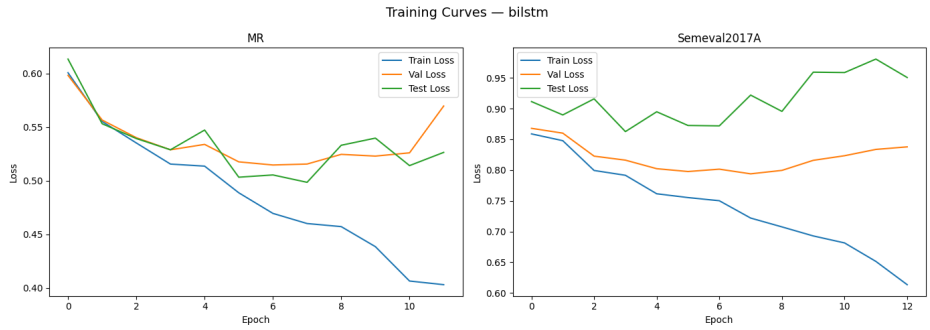


Figure 6: Καμπύλες εκπαίδευσης BiLSTM στο MR (αριστερά) και Semeval2017A (δεξιά).

```
Dataset: MR
  reps: 15
  avg_epochs_run: 12.9/50
  accuracy: 0.7458207452 +/- 0.0057445965
  recall: 0.7474589585 +/- 0.0062492491
  f1-score: 0.7454039576 +/- 0.0057527637

Dataset: Semeval2017A
  reps: 15
  avg_epochs_run: 12.6/50
  accuracy: 0.5886030609 +/- 0.0088734955
  recall: 0.5836741096 +/- 0.0081806495
  f1-score: 0.5697409504 +/- 0.0088096022
```

Το BiLSTM επιτυγχάνει 74.5% F1 στο MR και 57.0% στο Semeval, παραπλήσια με τον μονόδρομο LSTM (75.2% και 57.1%). Για τα σύντομα κείμενα των datasets αυτών η διαφορά είναι αμελητέα. Ίσως το BiLSTM θα παρουσίαζε σαφέστερο πλεονέκτημα σε μεγαλύτερες ακολουθίες, όπου η πληροφορία από τα πρώτα tokens θα είχε συμπιεστεί σημαντικά σε ένα μονόδρομο LSTM.

Ερώτημα 3: Self-Attention

3.1 Υλοποίηση και Αποτελέσματα

Ο μηχανισμός self-attention αντικαθιστά τη σειριακή επεξεργασία του LSTM με μια *παράλληλη* λειτουργία: κάθε λέξη "επικοινωνεί" ταυτόχρονα με όλες τις υπόλοιπες στην πρόταση, ώστε το αρχικό της embedding να αντικατασταθεί από μια σημασιολογικά εμπλουτισμένη αναπαράσταση βασισμένη στα συμφραζόμενα.

Την λειτουργία αυτή εκτελεί η κλάση `Head`, με τον τρόπο που απεικονίζεται παρακάτω:

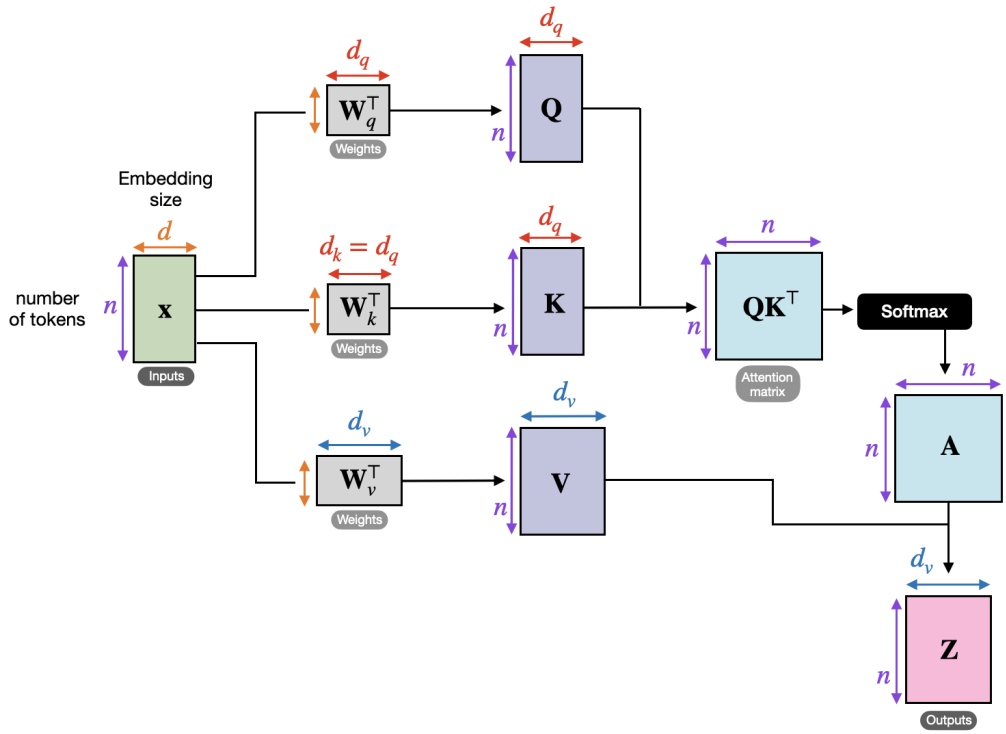


Figure 7: Διάγραμμα single-head self-attention. Η είσοδος $X \in R^{n \times d}$ είναι ο πίνακας αναπαραστάσεων όλων των tokens (embedding matrix). Τρεις γραμμικοί μετασχηματισμοί W^Q, W^K, W^V παράγουν τους πίνακες $Q, K, V \in R^{n \times d_k}$. Τα τελικά βάρη attention $A \in R^{n \times n}$ προκύπτουν ως $\text{softmax}(QK^T/\sqrt{d_k})$. Η έξοδος $Z = AV \in R^{n \times d_v}$ είναι η νέα, εμπλουτισμένη, αναπαράσταση κάθε token.

Η είσοδος X της `Head` κατασκευάζεται στο `forward` του `SimpleSelfAttentionModel` ως άθροισμα token embedding και position embedding κάθε θέσης:

```
# attention.py -- SimpleSelfAttentionModel.forward
tok_emb = self.token_embedding_table(x)          # (B, T, d)
pos_emb = self.position_embedding_table(
    torch.arange(T, device=x.device))             # (T, d)
x = tok_emb + pos_emb                            # X: (B, T, d)
x = x + self.sa(self.ln1(x))                     # weighted
   aggregation + residual
x = x + self.ffwd(self.ln2(x))                   # separate feed-
   forward block + residual
```

Ακολουθεί ένα feed-forward layer, και στη συνέχεια εφαρμόζεται masked average-pooling για να εξαχθεί αναπαράσταση ολόκληρης της πρότασης. Το τελικό γραμμικό classification layer ακολουθεί την ίδια λογική με τα προηγούμενα ερωτήματα.

```
Dataset: MR
  reps: 15
  avg_epochs_run: 16.8/50
  accuracy: 0.7241691843 +/- 0.0082332058
  recall: 0.7244958397 +/- 0.0082972549
  f1-score: 0.7240698838 +/- 0.0082220655

Dataset: Semeval2017A
  reps: 15
  avg_epochs_run: 20.7/50
  accuracy: 0.5914685770 +/- 0.0070428675
  recall: 0.5886292600 +/- 0.0098101850
  f1-score: 0.5622873579 +/- 0.0064423163
```

Το single-head self-attention επιτυγχάνει 72.4% F1 στο MR και 56.2% στο Semeval2017A. Και στα δύο datasets υπολείπεται του LSTM (75.2% και 57.1% αντίστοιχα). Αξίζει να σημειωθεί ότι το LSTM και BiLSTM αποδίδουν ιδιαίτερα καλά εδώ λόγω του σχετικά μικρού μήκους των προτάσεων στα datasets: η σειριακή επεξεργασία δεν υποφέρει από vanishing gradient σε τόσο σύντομες ακολουθίες, οπότε δεν αναδεικνύεται η αδυναμία της.

3.2 Queries, Keys, Values και Position Embeddings

Στην κλάση `Head` (`attention.py`), κάθε token παράγει τρία διανύσματα μέσω των εκπαιδεύσιμων γραμμικών προβολών W^Q, W^K, W^V, αντίστοιχα:

- **Query** (q, "τι πληροφορία ψάχνω;"): αντιπροσωπεύει αυτό που θέλει να μάθει το token από τα υπόλοιπα.
- **Key** (k, "τι πληροφορία προσφέρω;"): αντιπροσωπεύει αυτό που "διαφημίζει" το token ως περιεχόμενό του.
- **Value** (v, "ποιο είναι το πραγματικό μου περιεχόμενο;"): η πληροφορία που μεταφέρεται αν η "ερώτηση" ταιριάξει με το "κλειδί".

Όπως φαίνεται στο παραπάνω διάγραμμα, η έξοδος του head υπολογίζεται ως:

$$Z = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Ο παράγοντας $\sqrt{d_k}$ στον παρονομαστή συνίσταται για αριθμητική σταθερότητα: αν τα στοιχεία των q και k έχουν μέση τιμή 0 και διακύμανση 1, το εσωτερικό τους γινόμενο έχει διακύμανση d_k. Για μεγάλο d_k τα γινόμενα γίνονται αριθμητικά μεγάλα, ωθώντας το softmax σε περιοχές κορεσμού με μηδενικές σχεδόν κλίσεις.

Στην υλοποίησή μας επιλέχθηκε $d_k = d$, δηλαδή οι προβολές Q, K, V έχουν την ίδια διάσταση με τα embeddings.

Τα **position embeddings** (`position_embedding_table` στο `SimpleSelfAttentionModel`) είναι ένας εκπαιδεύσιμος πίνακας μεγέθους $T_{\max} \times d$. Ο μηχανισμός attention από μόνος του δεν έχει εγγενή έννοια θέσης: λειτουργεί σε ένα *σύνολο* διανυσμάτων και θα έδινε τα ίδια αποτελέσματα ανεξαρτήτως της σειράς τους. Προσθέτοντας το position embedding p_i στο token embedding e_i ($x_i = e_i + p_i$), το μοντέλο αποκτά πρόσβαση στη θέση κάθε token στην ακολουθία, επιτρέποντάς του να διαφοροποιήσει π.χ. "not good" από "good not".

Ερώτημα 4: MultiHead Attention

Αντί ενός μόνο attention head, χρησιμοποιούμε h heads παράλληλα, κάθε ένα με $d_k = d/h$ διαστάσεις (ώστε ο αριθμό των εκπαιδεύσιμων παραμέτρων να παραμένει περίπου σταθερός). Κάθε head μαθαίνει διαφορετικές σχέσεις μεταξύ tokens: ένα μπορεί να εστιάζει σε σχέσεις "γειτονικών" λέξεων, ένα άλλο σε μακρινές εξαρτήσεις, ένα τρίτο σε σημασιολογική ομοιότητα κλπ. Οι h στο πλήθος έξοδοι διάστασης d/h συνενώνονται γυρνώντας πίσω στην αρχική διάσταση (h x d/h = d) και επανα-προβάλλονται στην αρχική διάσταση αυτή μέσω εκπαιδεύσιμης προβολής W^O:

$$\text{MHA}(x) = [h_1 \,\|\, h_2 \,\|\, \cdots \,\|\, h_h]\, W^O$$

```
# attention.py -- MultiHeadAttention.__init__
self.heads = nn.ModuleList([Head(head_size, n_embd) for _ in range(
    num_heads)])
self.proj  = nn.Linear(n_embd, n_embd)                # W^O

# attention.py -- MultiHeadAttention.forward
out = torch.cat([h(x) for h in self.heads], dim=-1)  # concatenate heads
out = self.dropout(self.proj(out))                    # project back to d
```

Το υπόλοιπο `MultiHeadAttentionModel` παραμένει όμοιο με το `SimpleSelfAttentionModel`: χρησιμοποιούνται τα ίδια frozen embedding layer, position embeddings και το τελικό γραμμικό classification layer.

```
Dataset: MR
  reps: 10
  avg_epochs_run: 17.1/50
  accuracy: 0.7287009063 +/- 0.0112067897
  recall: 0.7295356774 +/- 0.0114888070
  f1-score: 0.7284604635 +/- 0.0111574351
```

```
Dataset: Semeval2017A
  reps: 10
  avg_epochs_run: 20.6/50
  accuracy: 0.5903451644 +/- 0.0059599274
  recall: 0.5895306980 +/- 0.0058248216
  f1-score: 0.5584019038 +/- 0.0081721759
```

Δοκιμάστηκαν $h \in \{2, 5, 10\}$ heads. Η καλύτερη επίδοση στο MR επιτεύχθηκε με $h = 2$ (72.8% F1), ενώ η αύξηση του αριθμού heads οδηγεί σε μικρή μείωση ($h = 5$: 72.5%, $h = 10$: 72.1%). Παρατηρούμε ότι, για σταθερό αριθμό εκπαιδεύσιμων παραμέτρων, προκύπτει ένας συμβιβασμός (trade-off) μεταξύ του πλήθους των κεφαλών (h) και της διάστασης προβολής (d_k). Καθώς το h αυξάνεται, η διάσταση d_k αναγκαστικά μειώνεται, ώστε να διατηρηθεί σταθερό το συνολικό μέγεθος του μοντέλου. Έτσι, ενώ ο μεγαλύτερος αριθμός κεφαλών επιτρέπει στο μοντέλο να εστιάσει ταυτόχρονα σε περισσότερες διαφορετικές συσχετίσεις μεταξύ των tokens, η αναπαράσταση αυτών των συσχετίσεων γίνεται σε έναν ολοένα και πιο περιορισμένο διανυσματικό χώρο (φτωχή αναπαράσταση). Όταν το d_k υποχωρεί κάτω από ένα κρίσιμο όριο, η εκφραστική ικανότητα κάθε κεφαλής εξασθενεί, οδηγώντας τελικά σε πτώση της συνολικής απόδοσης.

Ερώτημα 5: Transformer Encoder - BERT

Αρχιτεκτονική

Ο Transformer Encoder αποτελείται από πολλαπλά `Block`, κάθε ένα με:

1. MultiHead Self-Attention + residual connection + Layer Normalization
2. Feed-Forward Network ($d \to 4d \to d$) + residual connection + Layer Normalization

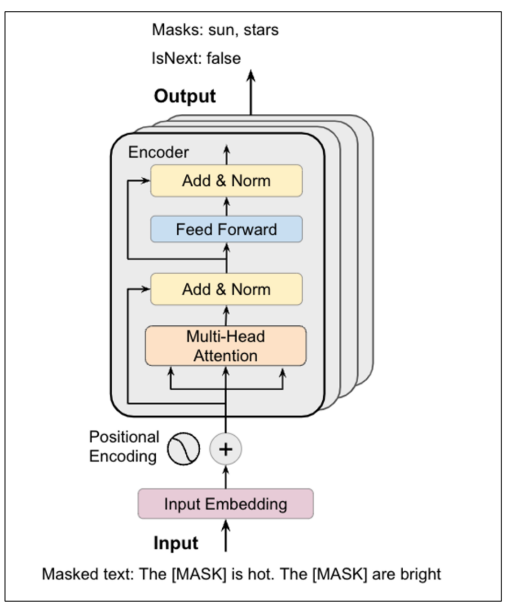


Figure 8: Αρχιτεκτονική Transformer Encoder

Η ιδέα πίσω από την αρχιτεκτονική είναι ανάλογη με αυτή ενός βαθιού νευρωνικού δικτύου: κάθε `Block` επεξεργάζεται την αναπαράσταση των tokens και την περνά στο επόμενο, επιτρέποντας στο μοντέλο να οικοδομεί διαδοχικά πιο αφηρημένες αναπαραστάσεις. Στο πρώτο επίπεδο, η self-attention συνδυάζει πληροφορίες από γειτονικά tokens για να αποτυπώσει τοπικές συντακτικές σχέσεις. Στα βαθύτερα επίπεδα, κάθε token έχει ήδη εμπλουτιστεί από τα συμφραζόμενα, οπότε το attention head αλληλεπιδρά με έννοιες υψηλότερου επιπέδου.

Η FFN που ακολουθεί εφαρμόζει μη-γραμμικό μετασχηματισμό πάνω στα αφηρημένα πλέον χαρακτηριστικά.

Τέλος, τα residual connections αντιμετωπίζουν το vanishing gradient problem, επιτρέποντας την εκπαίδευση βαθύτερων αρχιτεκτονικών.

Υπερπαράμετροι

Στην κλασική αρχιτεκτονική "Attention Is All You Need", οι default τιμές είναι: $d_{\text{model}} = 512$, $h = 8$ heads, $N = 6$ layers, $d_{ff} = 2048$, $d_k = d_v = d_{\text{model}}/h = 64$.

Στην υλοποίησή μας τα embeddings έχουν διάσταση $d = 50$, οπότε $d_{ff} = 4d = 200$ (ίδια αναλογία με το πρωτότυπο). Δοκιμάστηκαν $h \in \{2, 5, 10\}$ heads και $N \in \{2, 3, 5, 7, 10, 12\}$ layers μέσω grid search, καθώς και μια διαμόρφωση κοντά στις default τιμές: $h = 10$ (ο πλησιέστερος διαιρέτης του 50 στο $h = 8$) και $N = 6$. Παρατηρούμε ότι η ανεξέλεγκτη αύξηση του αριθμού των επιπέδων (N), και κατ' επέκταση της συνολικής χωρητικότητας του μοντέλου, δεν συνεπάγεται απαραίτητα βελτίωση των επιδόσεων. Αντιθέτως, πέρα από ένα ορισμένο βάθος, το μοντέλο καθίσταται υπερβολικά πολύπλοκο για τον όγκο των διαθέσιμων δεδομένων, με αποτέλεσμα να εμφανίζεται το φαινόμενο του overfitting. Σε τέτοιες περιπτώσεις, η ικανότητα γενίκευσης μειώνεται, καθώς το δίκτυο τείνει να απομνημονεύει τα δεδομένα εκπαίδευσης αντί να εξάγει ουσιαστικά γλωσσικά πρότυπα.

Αποτελέσματα

$h \setminus N$	2	3	5	7	10	12
2	.7271 .5602	.7288 .5676	.7295 .5681	.7270 .5581	.7252 .5586	.7278 .5565
5	.7260 .5658	.7313 .5656	.7185 .5679	.7262 .5592	.7282 .5584	.7169 .5582
10	.7241 .5668	.7232 .5635	.7183 .5649	.7322 .5653	.7226 .5577	.7223 .5615

Table 1: Grid search F1 αποτελέσματα (5 reps): MR (πάνω) / Semeval (κάτω) ανά κελί. Το χρώμα κωδικοποιεί τη μέση κανονικοποιημένη επίδοση και στα δύο datasets (μπλε = χαμηλότερο, κόκκινο = υψηλότερο). Η διαμόρφωση κοντά στις default τιμές (h=10, N=6) επιτυγχάνει F1 = 0.7200 (MR) / 0.5737 (Semeval).

Το grid search αναδεικνύει ότι ο h=10, N=7 επιτυγχάνει την υψηλότερη επίδοση στο MR *(73.2% F1)*, ενώ η κοντά-στις-default διαμόρφωση (h=10, N=6) κερδίζει στο Semeval *(57.4%)*, ωστόσο τα αποτελέσματα είναι παραπλήσια, ίσως βλέπαμε σημαντικότερες διαφορές αν χρησιμοποιούσαμε embeddings μεγαλύτερης διάστασης και περισσότερα δεδομένα εκπαίδευσης.

Ερώτημα 6: Pre-Trained Transformers (Zero-Shot)

Σε αυτό το ερώτημα αξιολογούμε προ-εκπαιδευμένα μοντέλα τύπου BERT σε zero-shot κατηγοριοποίηση: τα μοντέλα δεν εκπαιδεύονται εκ νέου στα δικά μας δεδομένα, αλλά χρησιμοποιούνται απευθείας για inference. Επιλέξαμε μοντέλα που έχουν ήδη γίνει fine-tuned σε sentiment analysis tasks, οπότε αρκεί να ζητήσουμε απευθείας την προβλεπόμενη κλάση.

Για το MR (κριτικές ταινιών) επιλέχθηκαν μοντέλα εκπαιδευμένα σε SST-2 ή παρόμοια general sentiment datasets, ενώ για το Semeval2017A (tweets) επιλέχθηκαν μοντέλα ειδικά εκπαιδευμένα σε Twitter, καθώς τα tweets έχουν αρκετά διαφορετική δομή (hashtags, mentions, slang, emojis).

MR Dataset

Μοντέλο	Accuracy	Recall	F1
DistilBERT-SST2	0.891	0.891	0.891
RoBERTa-large (siebert)	**0.924**	**0.924**	**0.924**
BERT-SST2 (textattack)	0.899	0.899	0.899

Semeval2017A Dataset

Μοντέλο	Accuracy	Recall	F1
twitter-roberta-base	**0.724**	0.723	**0.722**
bertweet-base	0.718	**0.730**	0.718
sentiment-roberta-large-3c	0.698	0.649	0.669

Σχολιασμός

Τα zero-shot αποτελέσματα είναι εντυπωσιακά: 92.4% F1 στο MR και 72.2% στο Semeval, ξεπερνώντας κατά πολύ τα μοντέλα Q1–Q5 (75.2% και 57.1% αντίστοιχα). Η διαφορά αυτή δεν οφείλεται αποκλειστικά στο μέγεθος των μοντέλων αλλά και στα πολύ μεγαλύτερα corpora που έχουν αξιοποιήσει ως pre-training.

Επίσης, όπως θα περίμενε κανείς, στο Semeval, η επιλογή Twitter-specific μοντέλων παράγει σημαντικά καλύτερα αποτελέσματα: Τα twitter-roberta-base (72.2%) και bertweet (71.8%) υπερτερούν αισθητά του sentiment-roberta-large-3c (66.9%), παρά τη μεγαλύτερη χωρητικότητα του τελευταίου.

Ερώτημα 7: Fine-Tuning Pre-Trained Transformers

Το fine-tuning ενημερώνει τα βάρη ενός προ-εκπαιδευμένου μοντέλου πάνω στα δικά μας δεδομένα. Αντί να ξεκινάμε από τυχαία αρχικοποίηση όπως στα Q1–Q5, το μοντέλο ξεκινά από μια ήδη ώριμη αναπαράσταση γλώσσας και χρειάζεται μόνο ελαφρά προσαρμογή για το νέο task. Χρησιμοποιούμε εδώ *base* μοντέλα (αντί για τα ειδικά sentiment-fine-tuned του Q6), εκπαιδεύοντάς τα για 5 epochs με batch size 32 και $\mathrm{lr} = 5 \times 10^{-5}$.

Ακολουθώντας την ίδια λογική domain matching του Q6, για το Semeval2017A επιλέχθηκαν μοντέλα Twitter-specific (twitter-roberta-base, bertweet), ενώ για το MR γενικά base μοντέλα (BERT, DistilBERT, RoBERTa).

MR Dataset

Μοντέλο	Epochs	Accuracy	Recall	F1
bert-base-uncased	5	0.849	0.840	0.848
distilbert-base-uncased	5	0.840	0.843	0.840
roberta-base	5	**0.881**	0.837	**0.875**

Semeval2017A Dataset

Μοντέλο	Epochs	Accuracy	Recall	F1
twitter-roberta-base	5	**0.703**	**0.718**	**0.704**
bertweet-base (vinai)	5	0.700	0.717	0.702
bert-base-cased	5	0.674	0.673	0.669

Σχολιασμός

Το fine-tuning επιτυγχάνει 87.5% F1 στο MR και 70.4% στο Semeval, υπολειπόμενο ελαφρώς των ήδη fine-tuned μοντέλων του Q6 (92.4% και 72.2%). Η διαφορά είναι αναμενόμενη: τα μοντέλα του Q6 εκπαιδεύτηκαν σε πολύ μεγαλύτερα και πιο αντιπροσωπευτικά sentiment datasets. Παρόλα αυτά, το χάσμα παραμένει μικρό, μόλις 5 epochs fine-tuning σε σχετικά μικρό training set φέρνουν ένα γενικό base model στο 87.5%, πολύ κοντά στο εξειδικευμένο.

Στο Semeval, τα Twitter-specific μοντέλα (twitter-roberta-base 70.4%, bertweet 70.2%) ξεπερνούν κατά ∼3% F1 το γενικό bert-base-cased (66.9%), επιβεβαιώνοντας ξανά ότι η αντιστοίχιση domain παραμένει σημαντικός παράγοντας ακόμα και μετά το fine-tuning.

Ζητούμενο 8 - Ανάλυση και Αξιολόγηση της Αλληλεπίδρασης με το LLM

Η αλληλεπίδραση με το ChatGPT σχετικά με τον κώδικα "GPT from scratch" του Andrej Karpathy ανέδειξε την ικανότητα των σύγχρονων μοντέλων να λειτουργούν ως εργαλεία επεξήγησης, αναλύοντας σύνθετες αρχιτεκτονικές σε πολλαπλά επίπεδα αφαίρεσης.

1. Επεξήγηση Κώδικα και Αρχιτεκτονικής

Η απόδοση του μοντέλου στην επεξήγηση υπήρξε εξαιρετικά ακριβής, συνδέοντας τον κώδικα PyTorch με τη θεωρία των Transformers.

- **Σημεία-Κλειδιά**: Το LLM διέκρινε ορθά τη ροή δεδομένων από το Character-level Tokenization (χρήση stoi/itos) προς τα Embeddings (Token + Positional).
- **Θεωρητική Σύνδεση**: Ερμήνευσε το Self-Attention ως μηχανισμό «επικοινωνίας» μεταξύ των tokens και το Feed-Forward Network (FFN) ως το στάδιο «επεξεργασίας/σκέψης» της πληροφορίας. Η ικανότητά του να εξηγεί τη σημασία των Residual Connections (x =

x + ...) για την αποφυγή του προβλήματος vanishing gradient ευθυγραμμίζεται πλήρως με τη θεωρία των βαθιών δικτύων.

2. Αξιολόγηση της Υλοποίησης

Στο στάδιο της αξιολόγησης, το LLM λειτούργησε ως «έμπειρος κριτικός», αναγνωρίζοντας τα trade-offs μεταξύ παιδαγωγικής σαφήνειας και υπολογιστικής απόδοσης.

- **Σημεία-Κλειδιά**: Εντόπισε ότι ο κώδικας, αν και ιδανικός για εκμάθηση των "first principles", στερείται κλιμακωσιμότητας.
- **Τεχνική Κριτική**: Ανέφερε συγκεκριμένες ελλείψεις έναντι των βιομηχανικών προτύπων (π.χ. GPT-4), όπως η απουσία Flash Attention και η χρήση απλού character-level encoding αντί για Byte-Pair Encoding (BPE), το οποίο θα μείωνε την πολυπλοκότητα $O(n^2)$ του attention mechanism σε μεγάλες ακολουθίες.

3. Ανασύνταξη (Refactoring) και Εκσυγχρονισμός

Το μοντέλο απέδειξε ότι μπορεί να αναβαθμίσει τον κώδικα ενσωματώνοντας σύγχρονες βελτιστοποιήσεις του PyTorch 2.0+.

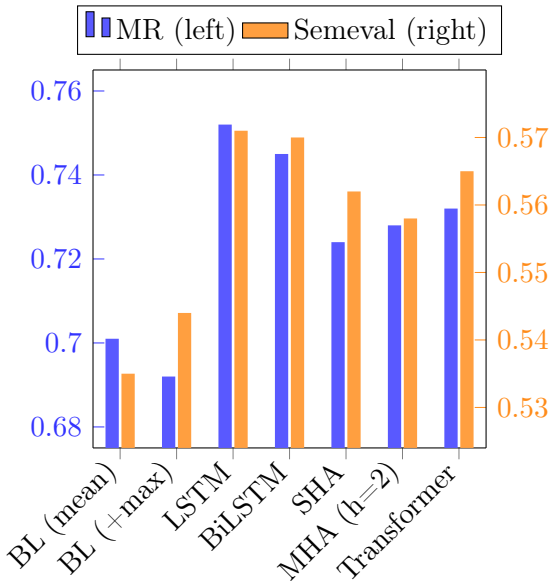
- **Σημεία-Κλειδιά**: Πρότεινε τη χρήση της scaled_dot_product_attention, η οποία είναι υπολογιστικά ανώτερη από τη χειροκίνητη υλοποίηση του Karpathy.
- **Βελτιώσεις**: Ενίσχυσε το modularity του κώδικα και πρόσθεσε μηχανισμούς σταθερότητας, όπως το Gradient Clipping, έννοιες κρίσιμες για την εκπαίδευση μεγάλων μοντέλων που εξετάζουμε στο εργαστήριο.

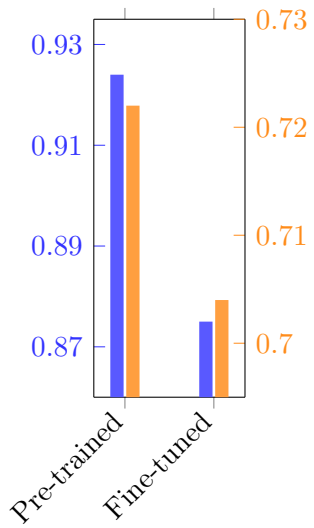
Συμπερασματικές Παρατηρήσεις

Η συνολική αλληλεπίδραση καταδεικνύει ότι τα LLMs δεν είναι απλώς γεννήτριες κώδικα, αλλά γνωστικοί βοηθοί ικανοί για βαθιά σημασιολογική ανάλυση. Το μοντέλο κατάφερε να γεφυρώσει το χαμηλό επίπεδο υλοποίησης (tensors/matrices) με το υψηλό επίπεδο αρχιτεκτονικής σχεδίασης. Η συνεισφορά του είναι ιδιαίτερα πολύτιμη στον εντοπισμό bottlenecks και στην εφαρμογή βέλτιστων πρακτικών best practices, καθιστώντας το αναπόσπαστο εργαλείο για την κατανόηση της εσωτερικής λειτουργίας των μοντέλων που μελετώνται στην Επεξεργασία Φυσικής Γλώσσας. Η συνομιλία με το μοντέλο (prompts και answers) είναι διαθέσιμα στο αρχείο $q8_chat_review$.

Συγκριτικός Πίνακας Αποτελεσμάτων

Μοντέλο	**MR F1**	**Semeval F1**
Baseline (mean)	.701	.535
Baseline (mean+max)	.692	.544
LSTM	.752	.571
BiLSTM	.745	.570
Self-Attention	.724	.562
MultiHead Attention (h=2)	.728	.558
Transformer Encoder (h=10, N=7)	.732	.565
Pre-trained (best)	.924	.722
Fine-tuned (best)	.875	.704
ChatGPT (zero-shot)†	1.000	1.000





† Αξιολογήθηκε σε μικρό υποσύνολο (40–60 δείγματα).

Bonus: LoRA Fine-Tuning

Το LoRA (Low-Rank Adaptation) είναι μια τεχνική parameter-efficient fine-tuning που παγώνει τα αρχικά βάρη του μοντέλου και εκπαιδεύει μόνο μικρές χαμηλής τάξης προσθήκες σε επιλεγμένα layers. Αντί να εκπαιδεύσουμε ολόκληρο τον πίνακα βαρών $\Delta W \in R^{d\times k}$, το LoRA τον αποσυνθέτει ως:

$$\Delta W = BA, \quad B \in R^{d\times r},\ A \in R^{r\times k},\ r \ll \min(d,k)$$

Τα αρχικά βάρη W_0 παραμένουν παγωμένα και η έξοδος κατά το forward pass είναι $h = W_0 x + \frac{\alpha}{r} BAx$. Ο A αρχικοποιείται με τυχαία Gaussian κατανομή και ο B με μηδενικά, έτσι ώστε $\Delta W = 0$ στην αρχή της εκπαίδευσης. Στην υλοποίησή μας $r = 8$ και $\alpha = 16$, οπότε ο κλιμακωτικός παράγοντας είναι $\alpha/r = 2$.

Υλοποίηση

Χρησιμοποιούμε τη βιβλιοθήκη HuggingFace PEFT και εφαρμόζουμε LoRA adapters σε όλα τα attention και FFN layers του `textattack/bert-base-uncased-SST-2`:

```
# bonus/finetune_lora.py
lora_config = LoraConfig(
    r=8,
    lora_alpha=16,
    lora_dropout=0.1,
    target_modules=[
        "query", "key", "value",                # attention projections
        "attention.output.dense",                # attention output
        "intermediate.dense", "output.dense",# FFN layers
    ],
    modules_to_save=["classifier"],             # fully trainable head
    bias="none",
)

model = get_peft_model(base_model, lora_config)
model.print_trainable_parameters()
# trainable params: 1,847,042 || all params: 151,268,354 || trainable
   %: 1.2%
```

Το `modules_to_save=["classifier"]` διασφαλίζει ότι ο classifier head παραμένει πλήρως εκπαιδεύσιμος: αν και μικρός, είναι ο μόνος πίνακας που αντιστοιχίζει τις αναπαραστάσεις του BERT στις κλάσεις του task, οπότε δεν έχει νόημα να τον περιορίσουμε σε low-rank.

Το learning rate ορίστηκε σε 2×10^{-4}, περίπου $10\times$ μεγαλύτερο από αυτό του Q7, καθώς οι LoRA adapters ξεκινούν από μηδενικά βάρη και χρειάζονται μεγαλύτερες αρχικές ενημερώσεις. Τα παγωμένα W_0 δεν επηρεάζονται, οπότε δεν υπάρχει κίνδυνος να διαταραχθεί η υπάρχουσα γλωσσολογική γνώση.

Αποτελέσματα

Μέθοδος	Trainable %	Accuracy	Recall	F1	Loss
Full fine-tuning	100%	0.900	0.876	0.898	0.597
LoRA ($r = 8$)	1.2%	**0.905**	0.876	**0.902**	**0.413**

Σχολιασμός

Το LoRA επιτυγχάνει 90.2% F1 με μόλις 1.2% εκπαιδεύσιμες παραμέτρους, ξεπερνώντας ελαφρώς το full fine-tuning (89.8%). Η διαφορά στο loss είναι σημαντική: 0.413 έναντι 0.597. Αυτό αποτελεί κλασικό σύμπτωμα ήπιου overfitting στο full fine-tuning. Από την άλλη, στην περίπτωση του LoRA ο περιορισμός χαμηλού rank δρα ως implicit regularizer, μειώνοντας τον αποτελεσματικό αριθμό ελεύθερων παραμέτρων και κρατώντας το μοντέλο εντός της αρχικής κατανομής του pre-training, η οποία αντιπροσωπεύει πολύ καλύτερα την πραγματική, γενική κατανομή, από ό,τι το περιορισμένο μας training set.